Week 3

Services, Parameters, and QoS

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

Lab 2 debrief

Whose square closed?

Nobody's did. Two independent reasons, and both matter later:

- Open-loop integration accumulates error with no correction
- The odometry you measured with is itself derived from a kinematic model — it cannot see slip, backlash, or the wheel it is not attached to

1. Four ways to talk

Choosing the right one

	Shape	Use when
Topic	one-way, many-to-many, continuous	Sensors, commands, state
Service	request/response, blocking, fast	A discrete question, answered quickly
Action	request/feedback/result, cancellable	Something slow that might be aborted
Parameter	named, typed, node-scoped	A value that tunes behaviour at runtime

Three common mistakes

- **A service for streaming data**
  Round-trip per sample. You will not make rate.
- **A topic for something needing acknowledgement**
  Publishing is fire-and-forget. You will not know whether anyone acted on it — or whether anyone was listening at all.
- **A service for something slow**
  It blocks and cannot be cancelled. That is what actions are for.

Nav2 uses all four

- `navigate_to_pose` — an **action**. Minutes long, must be cancellable
- `/goal_pose` — a **topic**
- Costmap tuning — **parameters**
- Lifecycle transitions — **services**

A footnote that will save you an hour:

`ros2 topic pub --once /goal_pose ...` **silently drops the goal.**
The publisher exits before discovery completes. Use `-r 2` under a `timeout`.

Another silent failure — written down because it cost someone real time.

2. Parameters

Live state, not startup configuration

```
self.declare_parameter('max_speed', 0.5)
value = self.get_parameter('max_speed').value
```

```
ros2 param get /governor max_speed
ros2 param set /governor max_speed 1.5     # takes effect immediately
```

- Everything in `mitt_controllers.yaml` and `nav2_params.yaml` arrives this way
- Which is why you can retune a planner without restarting it
- **And why a `param set` from a stray terminal changes your vehicle with nothing logged**

3. QoS — the silent one

Every endpoint carries a profile

Policy	Options
Reliability	`RELIABLE` / `BEST_EFFORT`
Durability	`TRANSIENT_LOCAL` / `VOLATILE`
History	`KEEP_LAST(n)` / `KEEP_ALL`

If the profiles are incompatible, they do not connect — and nothing says so.

The rule is asymmetric

RELIABLE publisher → BEST_EFFORT subscriber ✓

Reliable delivery is a **stronger** promise than best-effort demands.

BEST_EFFORT publisher → RELIABLE subscriber ✗

They never connect. No error. No warning. Nothing.

Conventions

Topic kind	Profile	Why
Sensor streams	`BEST_EFFORT`	A dropped scan beats a late one
Commands, state	`RELIABLE`	You need every one
`/robot_description`	`TRANSIENT_LOCAL`	Late joiners still receive it

4. Arbitration is not safety

What `twist_mux` does

Decides which **software** source reaches the controller.

- `cmd_vel_joy` → priority **100**
- `cmd_vel` (navigation) → priority **10**

A human at the sticks outranks the autonomy.

That is all it does.

What it cannot do

A software mux protects you from a **misbehaving software source**.

It cannot protect you from:

- the software itself dying
- the computer freezing
- the USB cable falling out

In each of those cases the mux is gone too.

The four real layers

Layer	Works if the firmware has crashed?
Hardware E-stop — cuts traction	**Yes**
Relay MUX — reverts to stock steering	**Yes**
Hardware RC signal MUX	**Yes**
RC serial override into the controller	No

Authority on this vehicle is **electrical and layered**, by design.

A safety mechanism that shares a failure mode with the thing it protects against is not a safety mechanism.

Today you build one anyway — and the objective is being able to say precisely why it must never be relied on.

Silent failures — running tally

Week	Failure	How it announces itself
1	`ROS_DOMAIN_ID` mismatch	Exit 0, empty stderr, 2 topics of 21
3	QoS incompatibility	Nothing whatsoever
4	Wrong sensor extrinsic	**Everything runs. The output is quietly wrong.**

Next week adds the worst kind: every component healthy, and the **system** wrong.

Lab 3 — A Speed Governor

1. A node with parameters; change the limit live with `ros2 param set`
2. A service to disable it — then argue why that is unacceptable in reality
3. Reproduce a QoS mismatch and diagnose it from `ros2 topic info --verbose`
4. Reflect: what do the two silent failures have in common?
5. **Break it on purpose** — make your own publisher best-effort

Due before Week 4 (10/05) — submit online, there is no class that day

Coming up

Week 4 (10/05) — ○ **No class,** 개천절 대체공휴일.
Self-study packet + **Lab 4**, both handed out today. Lab 4 due 10/12.

Week 5 (10/12) — Description to motion: URDF, TF2, RViz, Gazebo, `ros2_control`

Handed out today

- **Lab 4** — Ackermann kinematics; measure the turning radius
- **The Week 4 self-study packet** — read it over the break
- **The track preference form** — due **Friday 10/16**

Start thinking about which track you want:

Electrical & Firmware · Chassis & Measurement · Simulation & Validation · Software & Autonomy

Read Teams & Tracks before you rank them.